

SYSTEMATIC RADICALS

From algebraic numbers to certified radical towers

A constructive solver for exact polynomial `Root` objects

Prepared for Vladimir Reshetnikov

Version 1.0.0 • 8 September 2026

Abstract

An exact algebraic number need not become a radical expression when passed to a computer algebra system's usual radical converter. This is an algorithmic limitation, not a different notion of algebraic solvability. We develop a systematic replacement consisting of inexpensive structural reductions and a fully specified general construction. The latter builds an abstract splitting field, computes its automorphisms, passes to a cyclotomic base, and converts a prime-factor composition series into a radical tower by exact Lagrange resolvents. A separate checker validates positive certificates using rational linear algebra and polynomial identities, without reconstructing the Galois group. A coherent-branch theorem explains why principal radicals can represent the entire root set without combinatorial branch guessing.

The accompanying software contains a Wolfram Language interface with native fast paths and a self-contained Python/SymPy implementation of the general construction. Both examples in the motivating question have particularly small formulas: a reciprocal sextic reduces to a cubic, and a solvable quintic is a Dickson polynomial. Completeness is an unbounded algorithmic statement; default field-degree and time budgets deliberately allow an inconclusive resource-limit result.

Validation status. The Python implementation passed all 36 supplied regression tests under Python 3.13.5 and SymPy 1.14.0. They include exact tower certificates, selected-root checks, a nonsolvable polynomial, and negative protocol tests. Wolfram Language tests are supplied but were *not executed*: the available kernel connection failed. No claim of formal proof-assistant verification is made.

Contents

1	Problem, scope, and the meaning of success	2
2	What the experimental notebook contributes	3
3	Structural methods before Galois computation	3
3.1	Translation and power composition	3
3.2	Dickson polynomials	4
3.3	Weighted reciprocal reduction	4
4	The two original examples, explicitly	5
4.1	The sextic is a cubic followed by a quadratic	5
4.2	The quintic is a Dickson polynomial	6
5	Constructing the splitting field without numerical embeddings	6
5.1	Abstract fields and exact factorization	6
5.2	A finite primitive-element search by linear algebra	7
5.3	Why adjoining roots stops	7
6	Cyclotomic base change and an explicit Galois group	7
6.1	Recovering automorphisms from roots of the defining polynomial	8
7	Turning cyclic steps into radical generators	8
7.1	A deterministic nonzero Lagrange resolvent	9
7.2	Coordinates by Fourier projection	9
8	Coherent branches and selecting the intended root	10
8.1	The Wolfram acceptance condition	10
8.2	The Python selected-root comparison	11
9	Positive certificates and their independent checker	11
10	Completeness, termination, and cost	12
11	Software organization and use	13
11.1	Files and dependencies	13
11.2	Python API	13
11.3	Wolfram Language API	14
11.4	Data-only JSON and the worker boundary	15
12	Executed validation and remaining limitations	15
13	Conclusions and a development path	16
A	Algorithm-to-code map	17
B	Input provenance and reproducibility	17

1 Problem, scope, and the meaning of success

The motivating question asks why `ToRadicals` sometimes leaves behind a `Root` although a radical expression exists [1]. The official documentation explicitly acknowledges such cases and guarantees conversion through degree four [4]. The supplied notebook `FindExtension.nb` explores ways to discover suitable auxiliary fields. This article turns that exploratory idea into a specified construction, with explicit success, impossibility, and resource-limit outcomes.

Our basic input is a fixed number $\alpha \in \overline{\mathbb{Q}} \subset \mathbb{C}$. Its defining data include an exact polynomial and an embedding, such as a `Root` object. We exclude parameters, approximate coefficients, and transcendental constants whose algebraicity has not been established. The Wolfram interface first computes the minimal polynomial over $\mathbb{Q}$, so it also accepts exact algebraic expressions beyond a syntactically isolated `Root`. The Python polynomial entry point instead requires an irreducible polynomial over $\mathbb{Q}$; its selected-root entry point performs the factor selection itself.

Definition 1.1. A *radical expression over $\mathbb{Q}$* is a finite expression built from rational numbers by addition, subtraction, multiplication, division by nonzero quantities, and extraction of positive-integer roots. Complex intermediate values are allowed. Equivalently, we allow rational powers together with the field operations and $i = \sqrt{-1}$.

The implementation checks this grammar. A trigonometric value such as $2 \cos(2\pi/7)$ is not accepted as the final answer merely because it is algebraic. Roots of unity can, however, be written within the grammar:

$$\zeta_m = (-1)^{2/m} = \exp(2\pi i/m), \quad (1)$$

where the power on the left has the principal complex meaning. The exponential on the right is explanatory notation and does not occur in the exported expression.

It is important to solve the minimal polynomial of the selected number, not an arbitrary polynomial that happens to vanish there. For example,

$$(x - 10)(x^5 - x - 1)$$

has the easily expressible root 10 and other roots that are not expressible by radicals. Testing the whole product for solvability would incorrectly reject the easy root. Conversely, an irreducible polynomial cannot have one radical-solvable conjugate and another nonsolvable conjugate.

Theorem 1.2 (The criterion for a selected algebraic number). *Let f be the minimal polynomial of α over $\mathbb{Q}$ and let K be its splitting field. Then α has a radical expression over $\mathbb{Q}$ if and only if $\text{Gal}(K/\mathbb{Q})$ is solvable.*

Proof. The usual Galois solvability theorem states that a polynomial in characteristic zero is solvable by radicals exactly when its Galois group is solvable [2, Theorem 5.34]. To apply the criterion to one root, observe that if α belongs to a radical tower, adjoining the necessary roots of unity and all conjugates of the tower produces a finite normal extension with solvable Galois group. Every conjugate of α lies in that extension. Its splitting field therefore has a solvable group, being a quotient of a solvable group. The converse follows by constructing radicals for all roots. Sections 5–8 give that construction explicitly. $\square$

Three distinct questions must not be confused. Is the number solvable by radicals? Can the present run find a representation within its budget? Is a particular proposed representation the

selected root rather than a different conjugate? The package keeps these questions separate. A timeout is not a negative answer to the first question, and the identity $f(e) = 0$ is not by itself an answer to the third.

2 What the experimental notebook contributes

The archive supplied with the request contains one notebook, marked as created by Wolfram 15.0. We inspected its core definitions, including `findExtension`, `findExtension3`, `project`, `factor`, `solve`, `fuzz`, and the simplification helpers. The notebook is an experimental working document with many subsequent examples and saved outputs; the discussion here concerns those core algorithms, not a claim to have re-executed every notebook cell.

The extension searches eliminate coefficients from a proposed quadratic or cubic factor and look for a manageable coefficient field. This is a useful resolvent idea: coefficients of factors are symmetric functions of subsets of roots, and their stabilizers identify intermediate fields of a splitting field. The projection method examines real and imaginary parts of conjugates. The fuzzing method samples sums of conjugates, approximates them at high precision, and uses `RootApproximant` to seek a lower-degree description. These are plausible discovery techniques. They do not, by themselves, constitute a complete decision or construction procedure.

There are several concrete reasons to change the architecture. An unbounded random search has no negative stopping criterion. Failure to locate a quadratic or cubic auxiliary field says nothing about a needed cyclic extension of larger prime degree. A candidate factor must be checked by exact polynomial division and a selected root by exact algebraic equality; the notebook's uses of `PossibleZeroQ` and numerical recognition are not the acceptance condition used here. In addition, two early definitions refer to `Simplify'SimpifyCount`, whereas later definitions use `Simplify'SimplifyCount`. This spelling inconsistency is not a dependable complexity metric unless the first symbol was separately defined. Depending on an undocumented internal simplification symbol would be fragile even with consistent spelling.

The new implementation does not perpetuate a randomized search under a larger time limit. It replaces it by exact structural identities and then by a finite group-and-field construction. Simplicity of presentation is a separate optimization concern. A simple expression is preferred when a fast identity provides one; otherwise a correct straight-line radical program is more important than a cosmetically attractive but unverified formula.

A further distinction is between subfields of $\mathbb{Q}(\alpha)$ and subfields of its normal closure. A prime-degree root field has no proper intermediate fields, yet its normal closure can contain precisely the auxiliary fields needed to solve it. Restricting extension discovery to the root field is therefore not complete. The normal-closure construction below avoids that restriction.

3 Structural methods before Galois computation

For many examples the best solution is to avoid constructing a large splitting field. Both implementations recognize exact identities. Recognition uses rational coefficient comparisons, not numerical fitting.

3.1 Translation and power composition

A monic polynomial

$$f(x) = x^n + a_{n-1}x^{n-1} + \dots$$

can first be centered by $x = y + c$, $c = -a_{n-1}/n$. If all nonconstant exponents of the centered polynomial have common divisor $m > 1$, then

$$f(y + c) = g(y^m).$$

For every root b of g , the corresponding roots are

$$c + \zeta_m^k b^{1/m}, \quad 0 \leq k < m. \quad (2)$$

This includes shifted binomials and compositions of several such reductions. The equation is exact for every listed candidate. When the original input is irreducible in characteristic zero its roots are distinct. Repeated roots in an intermediate reduction do not create a correctness problem, although the minimal-polynomial preprocessing removes that issue for the top-level input.

Degree at most four is handled by the ordinary exact quadratic, cubic, and quartic routines. Their results still pass the radical-grammar check. The structural layer has a finite recursion bound; a miss simply transfers control to the general construction. The bound is not part of the completeness claim for the general layer.

3.2 Dickson polynomials

Define

$$D_0(t, a) = 2, \quad D_1(t, a) = t, \quad D_n(t, a) = tD_{n-1}(t, a) - aD_{n-2}(t, a).$$

Induction on n gives

$$D_n(u + a/u, a) = u^n + (a/u)^n \quad (u \neq 0). \quad (3)$$

Suppose the centered polynomial is $D_n(t, a) + b$. Set

$$v = \frac{-b + \sqrt{b^2 - 4a^n}}{2}, \quad u = v^{1/n}. \quad (4)$$

If the displayed v is zero, use the other quadratic root. The candidates are

$$c + \zeta_n^k u + \frac{a}{\zeta_n^k u}, \quad 0 \leq k < n. \quad (5)$$

Indeed, $v + a^n/v = -b$, and (3) applies. For a separable polynomial these give the full root set: a collision between different indices forces the degenerate situation $v^2 = a^n$, or equivalently the vanishing of the quadratic discriminant. The degenerate cases are excluded for the irreducible higher-degree inputs to which this path applies. The case $a = 0$ is already a power-composition case.

The parameter a is not searched. For $n \geq 3$ the coefficient of t^{n-2} in $D_n(t, a)$ is $-na$, so it determines a immediately. The full coefficient identity is then checked. This is a narrow but genuinely arbitrary-degree family of radical-solvable polynomials.

3.3 Weighted reciprocal reduction

Let a monic degree- $2m$ polynomial have coefficients

$$f(x) = \sum_{j=0}^{2m} c_j x^j.$$

Suppose a nonzero rational a satisfies

$$c_{m-k} = a^k c_{m+k} \quad (1 \leq k \leq m). \quad (6)$$

The recurrence above also gives

$$x^k + a^k x^{-k} = D_k(x + a/x, a).$$

Consequently

$$f(x) = x^m g(x + a/x), \quad g(t) = c_m + \sum_{k=1}^m c_{m+k} D_k(t, a). \quad (7)$$

After solving $g(t) = 0$, solve $x^2 - tx + a = 0$:

$$x = \frac{t \pm \sqrt{t^2 - 4a}}{2}. \quad (8)$$

No root at zero has been lost, since the constant coefficient in (6) is nonzero.

The implementation finds candidate a from the first nonzero ratio $c_{m-k}/c_{m+k} = a^k$, tests whether that ratio is an exact rational k th power, and verifies all of (6). For even k it considers both rational signs. It tries both the original and centered polynomial. This is not a complete search over arbitrary rational substitutions; it is an inexpensive, proved recognition method with a precise domain.

4 The two original examples, explicitly

4.1 The sextic is a cubic followed by a quadratic

For

$$f(x) = x^6 + x^4 - x^3 - x^2 - 1$$

there is the identity

$$f(x) = x^3 \left((x - x^{-1})^3 + 4(x - x^{-1}) - 1 \right). \quad (9)$$

Thus $t = x - x^{-1}$ satisfies $t^3 + 4t - 1 = 0$. Define

$$t = \sqrt[3]{\frac{9 + \sqrt{849}}{18}} - \sqrt[3]{\frac{\sqrt{849} - 9}{18}}. \quad (10)$$

Both cube-root radicands are positive, so there is no principal-versus-real cube-root ambiguity in this formula. If they are denoted u^3 and v^3 , then $u^3 - v^3 = 1$ and $uv = 4/3$. Hence $(u - v)^3 + 4(u - v) = 1$. The derivative $3t^2 + 4$ is positive on the real line, so this is the unique real root of the cubic. The two real roots of the sextic are therefore

$$\alpha_{\pm} = \frac{t \pm \sqrt{t^2 + 4}}{2}. \quad (11)$$

Their signs are respectively positive and negative. In particular,

$$\boxed{\text{Root}[\#^6 + \#^4 - \#^3 - \#^2 - 1 \ \&, 2] = \frac{t + \sqrt{t^2 + 4}}{2}}$$

with t from (10); numerically this is about 1.1306854. The other four roots follow by inserting the other two cubic roots into (11). The software uses the general reciprocal identity, not a hard-coded entry for this polynomial.

4.2 The quintic is a Dickson polynomial

The second example is

$$5x^5 - 25x^3 + 25x + 6 = 5 \left(D_5(x, 1) + \frac{6}{5} \right).$$

Let

$$u = \left(\frac{-3 + 4i}{5} \right)^{1/5}, \quad \zeta_5 = (-1)^{2/5}.$$

The five roots are

$$\beta_k = \zeta_5^k u + (\zeta_5^k u)^{-1}, \quad 0 \leq k < 5. \quad (12)$$

Since $|(-3 + 4i)/5| = 1$, these roots are real. The argument of the principal u lies between 0 and $\pi/5$, making β_0 the largest root. Thus the question's fifth real root can be written particularly simply as

$$\boxed{\beta_0 = \left(\frac{-3 + 4i}{5} \right)^{1/5} + \left(\frac{-3 - 4i}{5} \right)^{1/5}}. \quad (13)$$

Its approximate value is 1.80706. The occurrence of complex intermediates is allowed by our definition, even though all five outputs are real.

These derivations establish polynomial membership analytically. The selected root is then identified by its real ordering, and the supplied Python tests also check the correspondence using the library's root-separation comparison. Numerical values are for orientation, not the acceptance criterion.

5 Constructing the splitting field without numerical embeddings

Fast paths cannot cover every solvable Galois group. We now describe the implemented general fallback. The starting point is an irreducible monic $f \in \mathbb{Q}[x]$, of degree $n \geq 2$.

5.1 Abstract fields and exact factorization

A field is represented as

$$F = \mathbb{Q}[t]/(P(t)),$$

where P is irreducible. An element is a rational coefficient vector in the basis $1, t, \dots, t^{d-1}$, $d = \deg P$. We do not identify t with an approximate complex root. Addition, multiplication, inversion, equality, and polynomial evaluation take place in this quotient field.

Start with $P = f$ and factor f over F . If all factors are linear, the field already contains every root and is a splitting field. Otherwise choose a nonlinear irreducible factor $h(y) \in F[y]$, adjoin one root, and repeat. Exact number-field factorization is delegated to SymPy's algebraic-field polynomial arithmetic; the primitive-element change of coordinates is implemented here. The documented abstract algebraic-field interface allows a defining polynomial with a formal root symbol [6]. This avoids trying to build a primitive element by repeatedly comparing high-precision complex roots.

5.2 A finite primitive-element search by linear algebra

Suppose $F = \mathbb{Q}[t]/P$ has degree d , and $h(y)$ is monic irreducible over F of degree $m > 1$. The quotient $A = F[y]/h$ is a field of degree $N = dm$ over $\mathbb{Q}$. Use the basis

$$t^i y^j \quad (0 \leq i < d, \ 0 \leq j < m).$$

For $c = 1, 2, 3, \dots$, put $w = t + cy$ and form the rational matrix

$$B_c = [\text{vec}(1) \ \text{vec}(w) \ \dots \ \text{vec}(w^{N-1})]. \quad (14)$$

If B_c is nonsingular, solve for the coordinates of w^N and t in this basis of powers. Writing $w^N = \sum_{j=0}^{N-1} b_j w^j$ gives the new defining polynomial

$$P_{\text{new}}(W) = W^N - \sum_{j=0}^{N-1} b_j W^j.$$

The coordinate expression $t = T(w)$ also gives $y = (w - T(w))/c$. The implementation checks the old defining relation and the adjoined-root relation after this substitution.

Lemma 5.1. *The search for a nonsingular B_c terminates, and the resulting polynomial is the irreducible minimal polynomial of a primitive element of A .*

Proof. Since the characteristic is zero, A has N distinct embeddings into an algebraic closure. For any two different embeddings σ, τ , the equality

$$\sigma(t) + c\sigma(y) = \tau(t) + c\tau(y)$$

holds for at most one c , unless both images of t and y coincide. The latter would make the embeddings identical, because t, y generate A . Only finitely many c are therefore bad. For every other c , w has N distinct conjugates and degree N . Its first N powers form a basis, so B_c is invertible. Conversely, invertibility itself certifies that w has degree N . Its displayed degree- N relation must be its minimal polynomial. $\square$

5.3 Why adjoining roots stops

Every intermediate field embeds into a splitting field of f : it is generated by some roots of f , and adjoining a root of an irreducible factor over that field amounts to adjoining another root of f . Each proper adjunction increases the degree. The final degree is at most $n!$, and after finitely many adjunctions f splits. A splitting field of a separable polynomial is normal. This gives a finite construction, even though the resulting degree and coefficient heights may be very large.

The code chooses a factor of smallest available nonlinear degree, and uses a deterministic tie-breaker. This is a useful local heuristic, not a proof of minimum intermediate coefficient height. No particular complex ordering of roots is involved at this stage.

6 Cyclotomic base change and an explicit Galois group

Let K be the splitting field just constructed, and write $D = [K : \mathbb{Q}]$. Define

$$M = \text{rad}(D) = \prod_{p|D} p, \quad C = \mathbb{Q}(\zeta_M), \quad L = KC.$$

To construct L , factor the cyclotomic polynomial Φ_M over K . If it has a linear factor, a primitive M th root already belongs to K . Otherwise adjoin a root of an irreducible factor by the same quotient-field procedure. The resulting field contains all powers of that root, so Φ_M splits there. Both K and C are normal over $\mathbb{Q}$; hence L is normal too.

Only the squarefree product of the prime divisors of D is required, rather than a root of unity of order $D!$. The reason is that we will use a composition series with *prime* cyclic factors. This can still be an expensive base change: the degree of L is bounded by $D\varphi(M)$, not by D alone.

6.1 Recovering automorphisms from roots of the defining polynomial

Write $L = \mathbb{Q}[t]/P$ and $d = [L : \mathbb{Q}]$. Factor P over L . Normality means that its d roots $a_1, \dots, a_d$ all lie in L . They give all automorphisms

$$\sigma_j(t) = a_j, \quad \sigma_j\left(\sum_k q_k t^k\right) = \sum_k q_k a_j^k.$$

The implementation retains the automorphisms fixing the chosen primitive root z of Φ_M . They form

$$H = \text{Gal}(L/C), \quad |H| = d/\varphi(M).$$

It checks the expected cardinality and builds their multiplication table by exact substitution. A regular permutation representation of that table is then passed to the finite-group routines for solvability and composition series [7]. In particular, this is not a call to an unimplemented “general Galois oracle.” The group elements are actually constructed as field automorphisms.

For degrees through six a preliminary call to SymPy’s public `galois_group` can reject nonsolvable inputs cheaply. The documented second return value of that routine indicates containment in the alternating group; it is *not* a solvability flag [6]. The implementation uses the returned group’s solvability property. This low-degree optimization is not the degree range of the general algorithm.

Proposition 6.1. *H is solvable if and only if $G = \text{Gal}(K/\mathbb{Q})$ is solvable. Every prime divisor of $|H|$ divides M .*

Proof. Restriction identifies H with $\text{Gal}(K/K \cap C)$, a normal subgroup of G . The quotient is $\text{Gal}(K \cap C/\mathbb{Q})$ and is abelian, because it is a quotient of the abelian cyclotomic group. A group is solvable exactly when a normal subgroup and its quotient are solvable. Also $|H|$ divides $|G| = D$, proving the prime-divisor statement. $\square$

Thus a nonsolvable H is an actual impossibility result. Failure to construct H within a degree or time budget is not. When H is solvable, compute

$$H = H_0 \triangleright H_1 \triangleright \dots \triangleright H_s = 1, \quad H_{i-1}/H_i \simeq C_{p_i} \tag{15}$$

with p_i prime. Only successive normality is needed; the subgroups need not all be normal in H .

7 Turning cyclic steps into radical generators

Let

$$E_i = L^{H_i}.$$

The fixed fields give an ascending tower

$$C = E_0 \subset E_1 \subset \cdots \subset E_s = L, \quad [E_i : E_{i-1}] = p_i.$$

Choose $\sigma \in H_{i-1} \setminus H_i$. Its restriction to E_i generates the cyclic quotient of order $p = p_i$. Its order as an automorphism of all of L need not be p ; what matters is that σ^p is trivial on E_i . Put $\epsilon = z^{M/p} \in C$.

7.1 A deterministic nonzero Lagrange resolvent

For $b \in E_i$, consider

$$\mathcal{P}(b) = \sum_{j=0}^{p-1} \epsilon^{-j} \sigma^j(b). \quad (16)$$

Cyclic reindexing gives $\sigma(\mathcal{P}(b)) = \epsilon \mathcal{P}(b)$. Therefore, whenever $r = \mathcal{P}(b) \neq 0$,

$$r^p \in E_{i-1}, \quad E_i = E_{i-1}(r). \quad (17)$$

The second assertion follows because r is not fixed by σ and the relative degree is prime.

We need an actual finite search for b , not an instruction to “choose a generic element.” For the power basis of L , form

$$b_k = \sum_{h \in H_i} h(t^k), \quad 0 \leq k < d. \quad (18)$$

These elements span E_i over $\mathbb{Q}$. Indeed, the trace onto E_i is a surjective $\mathbb{Q}$ -linear map: on an element of E_i it is multiplication by $|H_i|$. Applying it to a basis of L therefore gives a spanning set.

Lemma 7.1. *At least one of $\mathcal{P}(b_0), \dots, \mathcal{P}(b_{d-1})$ is nonzero.*

Proof. The distinct automorphisms $1, \sigma, \dots, \sigma^{p-1}$ of E_i are linearly independent as maps with coefficients in E_i . For completeness, a minimal nonzero dependence $\sum_j a_j \tau_j(x) = 0$ for all x can be evaluated at ux and compared with $\tau_1(u)$ times the original dependence. Choosing u on which another τ_j differs from τ_1 gives a shorter nonzero dependence, a contradiction. Thus (16), whose coefficients are nonzero, is not the zero operator. It cannot vanish on a spanning set. $\square$

The code scans the finite list in (18), chooses the first nonzero resolvent, and rescales it by a nonzero rational to normalize its coefficient vector. It then checks exactly that H_i fixes r , that $\sigma(r) = \epsilon r$, and that every element of H_{i-1} fixes r^p . Rescaling affects the radicand by a rational p th power but changes neither the eigenvalue nor the generated field.

7.2 Coordinates by Fourier projection

Merely finding r_i is insufficient: one must express its radicand in earlier generators and express the requested roots in the completed tower. There is a particularly convenient coordinate formula. For $a \in E_i$ write

$$a = \sum_{j=0}^{p-1} c_j r^j, \quad c_j \in E_{i-1}.$$

Then

$$c_j = \frac{1}{p} \sum_{k=0}^{p-1} \sigma^k \left(\frac{a}{r^j} \right). \quad (19)$$

To verify this, substitute the expansion of a . The coefficient of $c_\ell r^{\ell-j}$ is the geometric sum $p^{-1} \sum_k \epsilon^{k(\ell-j)}$, equal to one for $\ell = j$ and zero otherwise. This also proves that each computed coefficient is in E_{i-1} .

Apply (19) recursively. At level zero, express an element of C in the rational basis $1, z, \dots, z^{\varphi(M)-1}$ by exact linear algebra. For every step this produces

$$r_i^{p_i} = A_i(z, r_1, \dots, r_{i-1}), \quad \alpha_j = B_j(z, r_1, \dots, r_s). \quad (20)$$

The coordinate expressions can be polynomials in the earlier generators with rational coefficients, with exponents below the corresponding relative degrees. The algorithm checks the reconstruction identity at each recursive step. Memoization avoids recomputing the same coordinates, but no simplification search is necessary for correctness.

8 Coherent branches and selecting the intended root

An abstract field generator is not born with a principal complex branch. It would be incorrect to derive several unrelated identities, take the principal root in each, and assume that all algebraic relations survive. For example, principal square roots do not obey $\sqrt{a}\sqrt{b} = \sqrt{ab}$ for all complex a, b . The remedy is a *single shared tower*, not a collection of independently simplified formulas.

Theorem 8.1 (Coherent-branch root-set theorem). *Suppose the exact relations in (20) describe the tower $C = E_0 \subset \dots \subset E_s = L$, with each step having degree p_i . Assign z the principal value $(-1)^{2/M}$. In order, assign r_i any one root of*

$$X^{p_i} - A_i(z, r_1, \dots, r_{i-1}).$$

Reuse that value everywhere r_i occurs. Then the evaluated expressions $B_1, \dots, B_n$ are exactly the distinct roots of f . In particular, always using the principal p_i th root is valid for the whole root set.

Proof. The initial assignment extends to an embedding of C into $\mathbb{C}$, because the assigned value is a primitive M th root of unity. Assume an embedding of E_{i-1} has been defined. Since $E_i = E_{i-1}(r_i)$ has degree p_i , the polynomial $X^{p_i} - A_i$ is its minimal polynomial. Applying the embedding to its coefficients preserves irreducibility over the image field, and mapping r_i to any root of the image polynomial extends the embedding to E_i . Inductively we obtain a field embedding $L \hookrightarrow \mathbb{C}$. It preserves all polynomial identities and cannot identify two distinct roots. The n distinct embedded roots of a degree- n polynomial are the full root set. $\square$

This result removes exponential enumeration of branch choices from the construction. It does *not* identify which output corresponds to the user's root index. That is a separate comparison after the root set is known. Nor does it justify a branch-unsafe transformation such as replacing the principal radical of a product by a product of principal radicals.

8.1 The Wolfram acceptance condition

The Wolfram interface uses approximate distances only to order candidates. It returns a candidate e only after

```
RootReduce[e - alpha] == 0
```

has succeeded, in addition to the radical-grammar check. A timeout or an unevaluated comparison is not accepted. This prevents accidentally returning a conjugate with the same minimal polynomial. No Python root index is assumed to agree with Wolfram's root ordering.

8.2 The Python selected-root comparison

The Python entry point `solve_root` uses zero-based `CRootOf` ordering. It obtains the minimal factor of that particular root, constructs its radical root set, and calls `Poly.same_root`. The latter is documented as a comparison between two roots of the supplied polynomial [8]. That precondition is important: it is supplied by the algebraic construction, not inferred from numerical proximity.

In the tested SymPy implementation, this comparison uses a root-separation bound and bounded-error evaluation rather than a fixed decimal tolerance. The root-set certificate is an exact rational-arithmetic object; the selected-index comparison additionally trusts SymPy's bounded-error evaluation contract. We do not describe that numerical implementation as an independently verified interval-arithmetic proof. Applications requiring a smaller numerical trusted base can replace this final comparison with certified complex-ball or rectangular root isolation, without changing the tower construction.

9 Positive certificates and their independent checker

The general construction returns more than an expression. Its certificate contains a rational defining polynomial P for the final field, the input polynomial f , the integer M , coordinate vectors for z , the resolvents r_i , and all roots of f , together with the coordinate expressions in (20). These data are serialized as integer strings and a small expression syntax tree.

The checker is independent of the field-search history and of the Galois-group computation. It performs the following checks in $F = \mathbb{Q}[t]/P$:

1. P is irreducible, the polynomial being certified is nonconstant, and, when supplied, it agrees with the caller's expected polynomial up to monic normalization.
2. The proposed z satisfies $\Phi_M(z) = 0$. Each nonzero proposed r_i satisfies its stated power identity, using only earlier generators.

3. The monomials

$$z^a r_1^{b_1} \cdots r_s^{b_s}, \quad 0 \leq a < \varphi(M), \quad 0 \leq b_i < p_i, \quad (21)$$

number exactly $\deg P$ and have linearly independent rational coordinate vectors.

4. The output coordinates evaluate to the supplied distinct root witnesses, their number is $\deg f$, and each witness satisfies f exactly.

Proposition 9.1 (Soundness of the positive checker). *If these checks succeed, evaluating the stated radical tower with coherent principal branches produces the full set of roots of the certified polynomial.*

Proof. Irreducibility of P ensures that the ambient quotient is a field. The cyclotomic relation gives a base field of degree $\varphi(M)$. Each power relation bounds the next relative degree by p_i . The full independence of (21) forces equality at every step: a smaller degree at any stage would

make the final monomial family dependent. Thus the power relations are genuine minimal polynomials for the successive generators. The branch argument of Theorem 8.1 applies. The exact output and root identities then give $\deg f$ distinct roots. $\square$

The checker need not prove that F is the smallest possible splitting field, or even reconstruct its automorphisms. A larger valid radical field containing all roots would be an equally sound positive certificate. It also does not certify an external root index: that comparison remains separate.

The general constructor supports `verify=True`, and the Wolfram bridge requests that check by default. Fast structural results are justified by their recognized identities and ordinary degree-at-most-four solvers; they do not currently carry this number-field certificate. A `NotSolvable` result relies on the exact group calculation, not on a separately exported negative certificate. These trust boundaries are deliberate and documented.

10 Completeness, termination, and cost

Theorem 10.1 (Unbounded constructive algorithm). *For an irreducible polynomial over $\mathbb{Q}$, the general procedure described above, with resource bounds removed and correct terminating exact arithmetic primitives, terminates. It either constructs radical expressions for all roots or determines that none of its roots has a radical expression over $\mathbb{Q}$.*

Proof. The splitting-field adjunction procedure terminates by Section 5; each primitive-element change terminates by Lemma 5.1. Cyclotomic adjunction is finite. Factoring the final defining polynomial and constructing the automorphism table are finite exact operations. Finite-group solvability testing and a composition series terminate. If the group is nonsolvable, Theorem 1.2 and Proposition 6.1 give the negative conclusion. Otherwise there are finitely many prime steps; each has a nonzero resolvent in the finite list of Lemma 7.1. Coordinate recursion decreases its level at every recursive call. The resulting tower gives the roots by Theorem 8.1. $\square$

This theorem is not a claim that every program run with the default settings will succeed. By default the Python backend limits a working field to degree 48 and requests a 120-second budget. These are engineering choices, not a mathematical degree limit on radical-solvable inputs. A field of large degree may appear even for a low-degree polynomial; a cyclotomic base change can make it larger still. Exact coefficients may grow rapidly during primitive-element changes, group construction, and resolvent coordinates.

The algorithm explicitly enumerates a group and constructs a full splitting field. It is consequently not a polynomial-time implementation in the input bit length. This should not be misrepresented as an inherent lower bound on radical solving. Landau and Miller proved polynomial-time solvability testing and a polynomial-time radical construction in straight-line-program form [3]. Implementing that substantially more sophisticated algorithm is a different engineering project. Here the priority is an explicit, auditably constructive reference algorithm with modest dependencies and useful fast paths. Large expanded formulas can also destroy the advantage of a compact straight-line representation, regardless of the underlying decision algorithm.

The finite-budget result `ResourceLimit` carries no solvability conclusion. A native-only or fast-only miss is `NotFound`. An expression that could not be matched to the requested root in time gives `VerificationFailed` in the Wolfram interface. None of these should be converted into `NotSolvable` by calling code.

11 Software organization and use

11.1 Files and dependencies

The archive contains the article source and PDF, `SystematicRadicals.wl`, the Python backend, runnable examples, a regression suite, the actual Python validation report, and saved positive certificates. The reference environment was Python 3.13.5 with SymPy 1.14.0 on Linux. The dependency file pins SymPy to that version because the implementation uses its exact algebraic-field representations. Future versions should be revalidated rather than silently assumed compatible. Python 3.10 or later is required by the source syntax.

No SageMath, PARI/GP, Mathematica installation, or network service is needed to run the Python implementation. The Wolfram native layer needs no Python for its fast paths; its general method invokes the bundled local backend.

```
cd SystematicRadicals
python -m pip install -r requirements.txt
python examples/quickstart.py
python tests/run_validation.py
```

11.2 Python API

The main all-roots routine accepts an irreducible polynomial:

```
from systematic_radicals import solve_polynomial, verify_certificate
import sympy as s
x = s.symbols("x")

answer = solve_polynomial(x**3 - 2, x,
    method="galois", max_field_degree=48,
    max_seconds=120, verify=True)
print(answer.assignments)
print(answer.outputs)
assert verify_certificate(answer.certificate, x**3 - 2)
```

Add the archive's `python` directory to `sys.path`, or use the example scripts, which locate it automatically. `assignments` is the ordered radical program; `outputs` contains expressions in its variables. `expanded_roots()` substitutes all assignments and can be expensive. `diagnostics` records field degrees, group orders, and prime steps.

The default `method="auto"` tries identities before the general method. `method="galois"` forces the general construction except for rational roots, making it useful for tests.

The setting `method="fast"` never constructs a splitting field. To request the ideal unbounded construction use

```
answer = solve_polynomial(f, x, method="galois",
    max_field_degree=None, max_seconds=None, verify=True)
```

This is an explicit opt-in to potentially extreme computation.

For a selected root, including an input polynomial with several irreducible factors, use

```
from systematic_radicals import solve_root
f = x**6 + x**4 - x**3 - x**2 - 1
```

```
positive_root = solve_root(f, 1, x) # zero-based CRootOf index
```

Do not transfer indices from Wolfram Language. Exceptions distinguish `InvalidInput`, `NotFound`, `NotSolvable`, `ResourceLimit`, and `CertificateError`. The Python library's time budget is cooperative inside construction: an individual SymPy factorization cannot be interrupted by a check that occurs only between operations. It does not bound all preprocessing or selected-root evaluation. The command-line worker described below provides the harder operational boundary.

11.3 Wolfram Language API

After placing the archive's directory at a convenient location:

```
Get["/path/to/SystematicRadicals/SystematicRadicals.wl"];
alpha = Root[#^6 + #^4 - #^3 - #^2 - 1 &, 2];
r = Radicalize[alpha, Method -> "Native"];
VerifyRadical[alpha, r]
```

`Radicalize` returns an expression or `Failure`. `RadicalReport` returns an association containing the expression, a pure-function minimal polynomial, the method, and diagnostics. Its success flag is set only after exact selected-root verification, except for an input already in the required radical grammar, whose identity is immediate.

The principal options are:

Option	Default	Meaning
<code>Method</code>	<code>Automatic</code>	Native methods, then Python fallback.
<code>"PythonExecutable"</code>	<code>Automatic</code>	Local interpreter path.
<code>"TimeLimit"</code>	120	Requested overall computation budget.
<code>"VerificationTimeLimit"</code>	30	Upper limit per exact candidate comparison.
<code>"MaxFieldDegree"</code>	48	Cap on the backend's working field degree.
<code>"VerifyCertificate"</code>	<code>True</code>	Check a general positive certificate.

`Method -> "Native"` forbids Python; `"Python"` skips the native layer but allows Python fast paths; `"Galois"` forces the general backend unless the input is already a radical expression. An explicit Python executable path is useful on Windows or with a virtual environment. The process is invoked as an argument list with `RunProcess`, not through shell string interpolation [5].

For an unbounded attempt set all three relevant bounds to `Infinity`:

```
Radicalize[alpha, Method -> "Galois",
  "MaxFieldDegree" -> Infinity,
  "TimeLimit" -> Infinity,
  "VerificationTimeLimit" -> Infinity]
```

Startup, shutdown, operating-system scheduling, and external-library calls mean that the Wolfram option is not a real-time deadline guarantee. Backend failure, protocol failure, mathematical nonsolvability, and inconclusive verification have different failure tags. The supplied `.wlt` file can be run with `TestReport`; those Wolfram tests have not been executed in this work.

11.4 Data-only JSON and the worker boundary

The backend can be called directly:

```
python python/systematic_radicals.py --json
```

It reads one JSON object from standard input. Coefficients are listed from highest to lowest degree, each as a numerator/denominator pair of decimal integer strings. For example:

```
{"coefficients": [[["1","1"],["0","1"],["0","1"],["-2","1"]],
  "method":"galois", "max_field_degree":48,
  "max_seconds":120, "verify":true, "certificate":true}
```

The response contains a status, diagnostics, ordered assignments, and a root list. Expressions use only the tags `Q`, `Ref`, `Add`, `Mul`, and `Pow`. References are zero-based indices into earlier assignments. The Wolfram decoder validates the tags and integer strings and never calls `ToExpression` on subprocess data. Invalid or forward references are rejected.

A separate spawned worker performs construction and certificate checking. Its parent waits for the requested time and terminates the worker on expiry. Process startup and shutdown add overhead; the timeout applies to the worker wait, not to every millisecond of the command's lifetime. A successful command exits with code zero; other statuses use exit code two. This design keeps an expensive exact factorization from indefinitely blocking a bounded command-line request, subject to ordinary operating-system process control.

12 Executed validation and remaining limitations

All 36 Python tests passed in the recorded run. The full run took about 81.38 seconds. The test log and machine-readable report are included in `validation`; the table below gives selected test durations, including the checks performed by each test, rather than claiming isolated algorithmic benchmarks.

Input or test	Method/check	Seconds
Original sextic, all roots	Reciprocal reduction, root matching	0.51
Original quintic, all roots	Dickson identity, root matching	0.03
$(x-3)^7-2$	Power composition, root matching	5.93
$x^6+8x^4+x^3+16x^2+8$	Weighted reciprocal, root matching	1.14
x^2-2	Forced tower, independent certificate	0.03
x^3-2	Forced tower, independent certificate	0.43
x^3-3x+1	Forced tower, independent certificate	0.30
x^4-10x^2+1	Forced tower, independent certificate	0.12
x^4-2	Forced nonabelian tower, certificate	22.17
x^3-x-1	Forced nonabelian tower, certificate	37.62
x^5-x-1	Exact nonsolvability	0.01

The forced general examples are significant even though several have elementary closed forms: they exercise the actual field-and-group path instead of allowing a cubic or quartic formula to conceal an unimplemented fallback. For x^3-x-1 , the splitting field has degree six, the cyclotomic compositum has degree twelve, and the stabilizer composition orders are 6, 3, 1. For x^4-2 , the corresponding orders are 8, 4, 2, 1. The latter is a nonabelian 2-group case; the former requires both a quadratic and a cubic cyclic step. Saved certificates permit repeating the independent positive checks without redoing the field search.

The suite also checks malformed inputs, nonrational coefficients, a root selected from a reducible polynomial, rational and complex radical grammar, field-degree limits, a hard worker timeout, altered radicands, duplicate root witnesses, and a certificate presented for the wrong polynomial. The tests of all-root matching rely on already established polynomial membership, as required by `same_root`. They are not a substitute for the certificate proof.

For the nonsolvable control there is a small independent explanation. Modulo two,

$$x^5 - x - 1 = (x^2 + x + 1)(x^3 + x^2 + 1),$$

and the factors are distinct and irreducible. Modulo three the polynomial is irreducible. Thus it is irreducible over $\mathbb{Q}$ and its transitive degree-five Galois group contains an element of cycle type $(2)(3)$. Cubing that element gives a transposition. A transitive group of prime degree is primitive; a primitive permutation group containing a transposition is the full symmetric group. One way to see the latter assertion is to take the graph whose edges are the conjugate transpositions: its connected components form a block system, so the graph is connected, and the edge transpositions generate the full symmetric group. Therefore the group is S_5 , which is not solvable. The program's recorded group order is 120, with derived subgroup of order 60.

There are substantial performance limitations. The general construction is coefficient-height sensitive and can be slow beyond small working fields. An exploratory forced construction for a cyclic quintic was stopped while an internal exact operation was still running; it is not counted as a successful test or included in the timing table. This illustrates why cooperative library checks and externally enforced worker bounds are distinct. No claim is made that the general path has been stress-tested across all solvable transitive groups, all degrees, or all branch-degenerate-looking expressions. The native Wolfram interface has only been statically inspected, not kernel-tested.

The code also does not optimize radical depth, the number of radicals, total expression size, or the size of the cyclotomic base beyond the squarefree choice. It does not implement arbitrary polynomial decomposition, arbitrary Tschirnhaus transformations, or a real-radical-only variant. These are absent optimizations or different specifications, not hidden assumptions in the completeness theorem for complex radicals over $\mathbb{Q}$.

13 Conclusions and a development path

A systematic solver needs more than a powerful simplifier. It needs a precise algebraic domain, a genuine solvability test, a construction that reaches every solvable case in principle, and a branch-aware test for the selected root. The supplied implementation separates those responsibilities.

For the motivating examples, exact structural identities produce small answers without expensive Galois arithmetic. When these identities fail, the general fallback actually constructs the normal field, its automorphisms, and its cyclic prime steps. The Lagrange projectors have a finite nonzero search, the coordinate recursion is explicit, and the resulting positive certificate is checkable without trusting the construction's group search. Coherent principal branches are enough to obtain the complete root set; exact root identification then handles the original `Root` object.

The most valuable future improvements are performance-oriented: smaller primitive elements, better intermediate-field choices, a compact program representation preserved across the Wolfram bridge, and a more sophisticated Galois implementation avoiding full regular representations. A separately checkable negative certificate and an independently certified root-index comparison would further reduce the trusted computational base. None of those is

silently claimed by version 1.0.0. What is supplied is a working exact reference construction, useful native fast paths, reproducible positive certificates, and explicit boundaries on the guarantees of a bounded run.

A Algorithm-to-code map

Component	Python implementation
Input normalization	<code>_poly, solve_root</code>
Structural identities	<code>_fast, dickson, _rational_roots_of_power</code>
Abstract field arithmetic	<code>NumberField</code>
Primitive-element adjunction	<code>NumberField.adjoin</code>
Splitting field	<code>_normal_closure</code>
Cyclotomic base and automorphisms	First part of <code>_tower</code>
Composition series and resolvents	Middle of <code>_tower</code>
Recursive Fourier coordinates	Nested <code>express</code> function in <code>_tower</code>
Straight-line radical output	<code>RadicalSolution, encode_expr</code>
Positive checker	<code>verify_certificate</code>
Selected root	<code>solve_root</code>
JSON protocol and bounded worker	<code>_request, _worker, main</code>

The Wolfram layer exposes two solving functions, `Radicalize` and `RadicalReport`, and two validation functions, `RadicalExpressionQ` and `VerifyRadical`. The Python backend is the executed reference implementation for the general algorithm; the Wolfram file is an interface and an independent implementation of the inexpensive structural layer.

B Input provenance and reproducibility

The supplied archive was inspected as data and was not executed. Its SHA-256 hash is

```
381e80382e8736aa00e18d8fd48c6f391b9ba0b15e80ff7621e7589147174930
```

The extracted `FindExtension.nb` has SHA-256

```
d70951fdb88128556ee7ea762652413b6f8b5e970c4437c2f3293d9b07eac007
```

The original notebook is not redistributed in this package. The new code is provided under the MIT license. The included article is provided under the license stated in the archive. Build the PDF from this directory with two runs of `pdflatex SystematicRadicals.tex`, or with `latexmk -pdf`.

References

- [1] Vladimir Reshetnikov. *Why is ToRadicals[] not able to handle all cases? Is there a workaround?* Mathematica Stack Exchange, question 34011, 2013; consulted 8 September 2026. <https://mathematica.stackexchange.com/questions/34011/why-is-toradicals-not-able-to-handle-all-cases-is-there-a-workaround>.

- [2] J. S. Milne. *Fields and Galois Theory*, version 5.10, 2022. In particular, the cyclic-extension and solvability results in Section 5. <https://www.jmilne.org/math/CourseNotes/FT.pdf>.
- [3] Susan Landau and Gary Lee Miller. Solvability by radicals is in polynomial time. *Journal of Computer and System Sciences* **30**(2) (1985), 179–208. <https://www.cs.cmu.edu/~glmiller/Publications/Papers/LaMi85.pdf>.
- [4] Wolfram Research. *ToRadicals—Wolfram Language Documentation*. Consulted 8 September 2026. <https://reference.wolfram.com/language/ref/ToRadicals.html>.
- [5] Wolfram Research. *RunProcess—Wolfram Language Documentation*. Consulted 8 September 2026. <https://reference.wolfram.com/language/ref/RunProcess.html>.
- [6] SymPy Development Team. *Number Fields*, SymPy 1.14.0 documentation. Consulted 8 September 2026. <https://docs.sympy.org/latest/modules/polys/numberfields.html>.
- [7] SymPy Development Team. *Permutation Groups*, SymPy 1.14.0 documentation. Consulted 8 September 2026. https://docs.sympy.org/latest/modules/combinatorics/perm_groups.html.
- [8] SymPy Development Team. *Polynomial Manipulation Module Reference*, in particular `Poly.same_root`, SymPy 1.14.0 documentation. Consulted 8 September 2026. <https://docs.sympy.org/latest/modules/polys/reference.html>.